

Memory Strategies for Multi-Agent Systems

Per-agent isolation, shared state, and structured outputs as contracts between agents

Source: <https://learn.microsoft.com/en-us/agent-framework/concepts/workflows/state>

Per-agent memory is the default

- Each agent owns its own AgentSession — conversation state is not automatically shared between agents in a workflow
- Group Chat's own docs are explicit about this: agents do NOT share one session instance, because different agent types may implement the session abstraction differently
- Instead, the orchestrator broadcasts each response to every other agent, so each agent's own session stays synchronized before its next turn
- The result looks like shared memory from the outside, but mechanically it's per-agent memory kept in sync

Shared memory: workflow state

```
# Private to the writing executor by default  
ctx.set_state("draft_answer", answer)  
  
# Any executor reading the SAME key sees the same value,  
# starting the next superstep after the write  
cached = ctx.get_state("draft_answer")
```

Use a workflow-state key when data needs to outlive one message hop — a running tally, a cache, a value more than one downstream executor needs.

Controlling how much context an agent sees

Full conversation (default)

- Each agent sees the previous agent's input *and* every response so far
- Preserves grounding, but context grows with every hop

`chain_only_agent_responses=True`

- Each agent sees only the immediately previous agent's response
- Useful for translation pipelines and progressive refinement, where earlier turns would just be noise

Structured outputs as contracts between agents

- Day 3 Part A used a typed `TriageResult{route, summary, needs_work_item}` as one agent's final response
- In a multi-agent workflow, a typed value is also how one agent hands work to the next — the type itself is the contract
- Example: the lab's Critic emits `Answer{summary, bullets, citations, confidence}` — the workflow's terminal output, and the shape every caller can rely on
- A Planner emitting a typed list of sub-questions for the Retriever to consume is the same pattern, one hop earlier in the pipeline

Why typed contracts matter more with multiple agents

- A free-text handoff between two agents is fragile — the receiving agent has to parse prose reliably every single run
- Every additional untyped hop in a chain is another place formatting drift can silently break the next step
- A validated type fails loudly (a validation error) instead of quietly (a misparsed string) — much easier to catch in a golden-set run (Module 5)

State isolation risk

- Reusing one workflow-builder, executor, or agent instance across unrelated tasks leaks state between them
- Agent threads/sessions persist across workflow runs by default — useful for continuity within one task, a real risk across different ones
- Mitigation: wrap construction in a helper function so every run gets fresh executor and agent instances
- This is a "memory contract" failure as much as it's a "failure mode" — Module 6 revisits it from the guardrails angle

Choosing a memory strategy

You need...	Reach for...
Every agent to see the full running conversation	Default per-agent memory + orchestrator broadcast (Group Chat)
Only the immediately previous step's output	<code>chain_only_agent_responses=True</code> (Sequential)
A value that outlives one message hop	Workflow state with a shared key
A reliable, parseable handoff between two agents	A typed structured output, not free text
Fresh state on every run	Build executors/agents inside a helper function, not module-level singletons

Takeaways

- ✓ Agents don't automatically share memory — a workflow's orchestrator (or your own state) is what keeps them synchronized.
- ✓ Choose full-context or response-only chaining deliberately; it's a real trade-off between grounding and noise.
- ✓ Use workflow state for anything that needs to outlive a single message hop.
- ✓ Treat structured outputs as contracts between agents, not just response formatting — the lab's Critic→Planner conditional edge in Module 3 routes on exactly this kind of typed value.
- ✓ Build fresh executor/agent instances per run — reused instances leak state across unrelated tasks.